

Semana 1 — Clase Práctica en Laboratorio

Problemas de Algoritmos Secuenciales con Variables, Expresiones y Tipos de Datos

Introducción

Los problemas que siguen están diseñados para ejercitar la construcción de **algoritmos secuenciales** aplicando la **metodología de Pólya** (Análisis → Diseño → Codificación → Evaluación) y el **modelo de programa** del curso (Sección Declarativa + Sección Algorítmica: Prólogo, Resolución, Epílogo). Cada problema trabaja con tipos de datos primitivos (`int`, `float`, `str`), operaciones aritméticas y funciones de entrada/salida.

Para cada problema se proporcionan: el **enunciado**, las **fórmulas matemáticas** necesarias, **ejemplos numéricos** que sirven como casos de análisis y prueba, y **orientaciones** para la resolución. Se espera que cada estudiante desarrolle la solución completa siguiendo las fases de Pólya.

Recordatorio: Modelo de programa del curso

Sección Declarativa (docstring): descripción, casos de análisis y prueba, síntesis de casos, recursos (variables con nombre y dominio). Sección Algorítmica: 1) Prólogo (presentación al usuario), 2) Resolución (entrada de datos, cálculos), 3) Epílogo (exhibición de resultados).

Problema 1: Conversión de temperaturas

Enunciado

Escribir un programa en Python que solicite al usuario una temperatura expresada en **grados Celsius** y calcule su equivalente en **grados Fahrenheit** y en **Kelvin**. El programa debe mostrar ambos resultados con dos decimales de precisión.

Fundamento matemático

Las escalas de temperatura se relacionan mediante las siguientes fórmulas:

Celsius a Fahrenheit

La escala Fahrenheit, propuesta por Daniel Gabriel Fahrenheit en 1724, define el punto de congelación del agua en 32°F y el de ebullición en 212°F. La conversión desde Celsius requiere una transformación lineal:

$$F = C \times 9/5 + 32$$

donde **F** es la temperatura en Fahrenheit y **C** es la temperatura en Celsius. El factor 9/5 (= 1,8) refleja que un grado Fahrenheit es más pequeño que un grado Celsius, y el desplazamiento de 32 ajusta el origen de la escala.

Celsius a Kelvin

La escala Kelvin es la unidad de temperatura del Sistema Internacional. Su cero absoluto (0 K) corresponde a la ausencia teórica de energía térmica. La conversión es un simple desplazamiento:

$$K = C + 273,15$$

donde **K** es la temperatura en Kelvin. Nótese que el tamaño del grado es idéntico en ambas escalas; solo cambia el punto de referencia.

Ejemplos (casos de análisis y prueba)

Celsius (°C)	Fahrenheit (°F)	Kelvin (K)	Significado físico
0.00	32.00	273.15	Congelación del agua
100.00	212.00	373.15	Ebullición del agua
36.50	97.70	309.65	Temperatura corporal típica
-40.00	-40.00	233.15	Coincidencia Celsius-Fahrenheit
-273.15	-459.67	0.00	Cero absoluto (caso límite)

Orientaciones para la resolución

Análisis: Identificar la entrada (un valor `float` para Celsius), las dos fórmulas y las salidas esperadas. Verificar con los ejemplos proporcionados, prestando especial atención al caso **-40 °C** (donde ambas escalas coinciden) y al cero absoluto.

Diseño: Variables: `celsius` (`float`, entrada), `fahrenheit` (`float`, resultado), `kelvin` (`float`, resultado). Las fórmulas se traducen directamente a expresiones aritméticas.

Codificación: Recordar separar la lectura (`input()`) de la conversión de tipo (`float()`) en dos líneas distintas, como recomienda el estilo del curso.

Evaluación: Ejecutar el programa con cada fila de la tabla de ejemplos y comparar los resultados con los valores esperados.

Reflexión: precisión de punto flotante

Al operar con `9/5`, Python evalúa la división real (1.8). Sin embargo, los números `float` tienen precisión limitada (~15 dígitos, IEEE 754 doble). Verificar que los resultados coincidan con la tabla dentro de la tolerancia esperada. ¿Qué ocurriría si se usara la división entera `//` en lugar de la real `/`?

Problema 2: Índice de Masa Corporal (IMC)

Enunciado

Escribir un programa en Python que solicite al usuario su **peso en kilogramos** y su **estatura en metros**, calcule el **Índice de Masa Corporal (IMC)** y muestre el resultado con dos decimales de precisión.

Fundamento matemático

El IMC (o índice de Quetelet) es una medida estadística que relaciona el peso con la estatura de una persona. Fue definido por el matemático belga Adolphe Quetelet en 1832 y se utiliza ampliamente como indicador de composición corporal:

$$\text{IMC} = \text{peso} / \text{estatura}^2$$

donde **peso** se expresa en kilogramos (kg) y **estatura** en metros (m). El resultado se expresa en kg/m². En Python, el operador de potenciación es `**`, por lo que la estatura al cuadrado se expresa como `estatura ** 2`.

Ejemplos (casos de análisis y prueba)

Peso (kg)	Estatura (m)	IMC (kg/m²)	Observación
70.0	1.75	22.86	Caso típico
55.0	1.60	21.48	Caso típico
95.0	1.80	29.32	Caso con IMC elevado
50.0	1.50	22.22	Valores redondeados
80.0	2.00	20.00	Cálculo exacto (sin decimales residuales)

Verificación manual del primer ejemplo

Paso a paso para el caso peso = 70,0 kg, estatura = 1,75 m:

```
estatura² = 1.75 ** 2 = 3.0625
IMC = 70.0 / 3.0625 = 22.857142857...
Redondeado a 2 decimales: 22.86
```

Orientaciones para la resolución

Análisis: Dos entradas de tipo `float` (peso, estatura), una fórmula, una salida. Considerar: ¿qué ocurriría si la estatura fuera 0? (error de división por cero, tema que se profundizará con estructuras condicionales en la Semana 3).

Diseño: Variables: `peso` (`float`), `estatura` (`float`), `imc` (`float`). El cálculo se resuelve con una única expresión aritmética.

Codificación: Prestar atención a la precedencia de operadores: `peso / estatura ** 2` es correcto porque `**` tiene mayor precedencia que `/`. Sin embargo, es recomendable documentar esta decisión con un comentario.

Conexión con Arquitectura

El operador `**` se traduce en una secuencia de multiplicaciones en la ALU. A nivel de lenguaje de máquina, la potenciación no suele ser una instrucción nativa: el intérprete la implementa mediante operaciones más simples. Esto conecta con el concepto de abstracción de la clase teórica.

Problema 3: Fecha y hora como número fraccionario

Enunciado

Escribir un programa en Python que solicite al usuario los componentes de una **fecha y hora** por separado: **día** (entero), **mes** (entero), **año** (entero), **hora** (entero, 0–23) y **minutos** (entero, 0–59), y los combine en un único **número en punto flotante**. La **parte entera** codifica la fecha (año, mes, día) y la **parte fraccionaria** representa la **fracción del día** transcurrida según la hora y los minutos. El programa debe mostrar tanto la fecha y hora en formato legible como su representación numérica.

Fundamento matemático

La idea consiste en codificar un instante temporal completo como un único número `float`. La construcción se realiza en dos etapas, cada una con un criterio de diseño distinto:

Parte entera: la fecha (codificación posicional)

$$\text{parte_entera} = \text{año} \times 10000 + \text{mes} \times 100 + \text{día}$$

Esta codificación, inspirada en el formato **ISO 8601**, ubica el año en las decenas de millar, el mes en las centenas y el día en las unidades. Su propiedad clave es que el **orden numérico coincide con el orden cronológico**: si `fecha_A < fecha_B`, entonces A es anterior a B.

Parte fraccionaria: la fracción del día transcurrida

Para incorporar la hora al mismo número, se expresa el momento del día como una **fracción del día completo**. Un día tiene 24 horas y cada hora tiene 60 minutos, por lo que un día tiene $24 \times 60 = 1440$ minutos. La fracción del día transcurrida se calcula como:

$$\text{fraccion_dia} = \text{hora} / 24 + \text{minutos} / (24 \times 60)$$

o equivalentemente, factorizando:

$$\text{fraccion_dia} = \text{hora} / 24 + \text{minutos} / 1440$$

¿Por qué esta representación y no una codificación posicional? La clave está en que la fracción del día produce valores **proporcionales al tiempo real transcurrido**. Medianoche es

exactamente 0, mediodía es exactamente 0,5 y las 18:00 son exactamente 0,75. Esto significa que la **distancia numérica entre dos instantes es proporcional a la distancia temporal**, una propiedad que una codificación meramente posicional (como HHMM/10000) no tiene: en esa codificación, la distancia de 12:58 a 12:59 sería distinta a la de 12:59 a 13:00, aunque ambos intervalos representan exactamente un minuto.

Además, la fracción del día está siempre en el intervalo $[0, 1)$: el valor máximo posible es $23/24 + 59/1440 \approx 0,99931$, que es estrictamente menor que 1. Esto garantiza que la parte fraccionaria **nunca altera la parte entera**.

Fórmula completa

$$\text{fecha_hora} = \text{parte_entera} + \text{fraccion_dia}$$

Es decir, expandiendo:

$$\text{fecha_hora} = (\text{año} \times 10000 + \text{mes} \times 100 + \text{día}) + \text{hora} / 24 + \text{minutos} / 1440$$

Nótese que esta fórmula produce naturalmente un resultado de tipo `float` en Python, puesto que las divisiones `/ 24` y `/ 1440` siempre devuelven valores de punto flotante (a diferencia de la división entera `//`). Es precisamente la inclusión de la hora lo que **justifica y da sentido** a la representación en punto flotante: sin la parte horaria, un entero sería suficiente.

Valores de referencia de la fracción del día

Antes de los ejemplos completos, conviene internalizar algunos valores característicos de la fracción del día:

Hora:Minutos	hora/24	min/1440	Fracción del día	Interpretación
00:00	$0/24 = 0$	$0/1440 = 0$	0.0	Medianoche (inicio del día)
06:00	$6/24 = 0.25$	$0/1440 = 0$	0.25	Un cuarto del día
12:00	$12/24 = 0.5$	$0/1440 = 0$	0.5	Mediodía (medio día exacto)
18:00	$18/24 = 0.75$	$0/1440 = 0$	0.75	Tres cuartos del día
23:59	$23/24 \approx 0.9583$	$59/1440 \approx 0.0410$	≈ 0.99931	Casi medianoche (< 1)

Ejemplos completos (casos de análisis y prueba)

Día	Mes	Año	Hora	Min	Resultado (float)	Observación
15	8	2025	14	30	20250815.6042...	$14:30 = 60,42\%$ del día
1	1	2000	0	0	20000101.0	Medianoche, inicio de milenio

Día	Mes	Año	Hora	Min	Resultado (float)	Observación
31	12	1999	23	59	19991231.9993...	Un minuto antes del 2000
9	3	2026	8	15	20260309.3438...	Fecha actual, 08:15
25	12	2025	12	0	20251225.5	Mediodía de Navidad
29	2	2024	0	1	20240229.0007...	Año bisiesto, 00:01

Verificación manual del primer ejemplo

Paso a paso para día = 15, mes = 8, año = 2025, hora = 14, minutos = 30:

```
# Parte entera (fecha)
anio * 10000 = 2025 * 10000      = 20250000
mes * 100    = 8 * 100          = 800
dia          = 15
parte_entera = 20250000 + 800 + 15 = 20250815

# Parte fraccionaria (fracción del día)
hora / 24      = 14 / 24 = 0.58333...
minutos / 1440 = 30 / 1440 = 0.02083...
fraccion_dia   = 0.58333... + 0.02083... = 0.60417...

# Resultado
fecha_hora = 20250815 + 0.60417... = 20250815.60417...

# Verificación de sentido: 14:30 son 14.5 horas de 24,
# 14.5 / 24 = 0.60417..., o sea el 60,42% del día. ✓
```

Verificación del quinto ejemplo (mediodía)

Paso a paso para día = 25, mes = 12, año = 2025, hora = 12, minutos = 0:

```
parte_entera = 2025*10000 + 12*100 + 25 = 20251225
fraccion_dia = 12/24 + 0/1440 = 0.5 + 0.0 = 0.5
fecha_hora   = 20251225 + 0.5 = 20251225.5
```

Mediodía es exactamente la mitad del día, lo que produce una fracción de 0,5 exacta. Esta **correspondencia intuitiva** entre el valor numérico y el significado físico es una ventaja de la representación como fracción del día.

Verificación del tercer ejemplo (caso límite)

Paso a paso para día = 31, mes = 12, año = 1999, hora = 23, minutos = 59:

```
parte_entera = 1999*10000 + 12*100 + 31 = 19991231
fraccion_dia = 23/24 + 59/1440
              = 0.95833... + 0.04097...
              = 0.99931...
```

```
fecha_hora = 19991231 + 0.99931... = 19991231.99931...
```

Este instante representa un minuto antes de la medianoche del año 2000. Comparémoslo con el segundo ejemplo (medianoche del 1/1/2000): $19991231.9993... < 20000101.0$, confirmando que el **orden numérico preserva el orden cronológico** incluso al cruzar un cambio de año. Además, el valor $0.99931...$ es casi 1 pero no llega: faltan exactamente $1/1440 \approx 0.00069$ para completar el día (un minuto de 1440).

Orientaciones para la resolución

Análisis: Cinco entradas de tipo `int` (día, mes, año, hora, minutos), dos fórmulas intermedias (parte entera y fracción del día), dos salidas (fecha/hora legible y representación numérica). Pregunta clave: ¿la fracción del día es siempre menor que 1? Sí, porque el máximo posible ($23/24 + 59/1440 \approx 0.999$) nunca alcanza 1. Verificar también con mediodía (fracción = 0,5 exacto) y medianoche (fracción = 0,0 exacto).

Diseño: Variables: `dia`, `mes`, `año`, `hora`, `minutos` (`int`, entradas); `parte_entera` (`int`, cálculo intermedio); `fraccion_dia` (`float`, cálculo intermedio); `fecha_hora` (`float`, resultado). Descomponer el cálculo en pasos intermedios explícitos mejora la legibilidad y facilita la depuración. Nótese que `año` es un identificador válido en Python (acepta la ñ), pero se puede preferir `anio` por compatibilidad con teclados.

Codificación: Las divisiones `hora / 24` y `minutos / 1440` son las operaciones que producen el tipo `float`. Atención: la constante 1440 puede calcularse como `24 * 60` directamente en la expresión, lo que la hace más legible y autoexplicativa. Practicar la salida formateada mostrando la fecha/hora legible con f-strings (por ejemplo, `f'{dia}/{mes}/{anio} {hora}:{minutos:02d}'`). El especificador `:02d` asegura que los minutos siempre se muestren con dos dígitos (por ejemplo, `08:05` en lugar de `8:5`).

Evaluación: Verificar que la comparación numérica preserva el orden cronológico. Comprobar que el resultado del tercer ejemplo ($19991231.999...$) es menor que el del segundo (20000101.0). Verificar los casos de medianoche (fracción = 0,0) y mediodía (fracción = 0,5). Comprobar que la fracción del día tiene sentido intuitivo: las 14:30 están pasado el 60% del día, no al 14%.

Conexión con Tipos de Datos: ¿por qué float y no int?

En la versión que solo codifica la fecha (sin hora), un entero basta: 20250815 es un `int` que no pierde información. Al agregar la hora necesitamos una parte fraccionaria genuina, lo que obliga a usar `float`. Esto ilustra concretamente la razón de existencia del tipo `float`: representar valores que no son enteros. Además, la precisión de IEEE 754 doble (~15 dígitos significativos) es más que suficiente: la parte entera consume 8 dígitos, dejando ~7 para la fracción del día, lo que permite discriminar instantes con resolución muy por debajo del minuto.

Reflexión: ¿por qué fracción del día y no codificación posicional?

Una alternativa sería codificar HHMM como entero y dividir por 10000, obteniendo por ejemplo 0,1430 para las 14:30. Sin embargo, esa codificación no es proporcional al tiempo: el salto de 12:59 a 13:00 sería numéricamente distinto al de 12:58 a 12:59 (0.0041 vs 0.0001), aunque ambos representan exactamente un minuto. En cambio, la fracción del día asigna a cada minuto la misma distancia numérica: $1/1440 \approx 0.000694$.

Esta proporcionalidad es la misma propiedad que usan formatos profesionales como el número de serie de fecha de las hojas de cálculo (Excel, LibreOffice Calc).

Problema 4: Término de Fibonacci mediante el número de oro

Enunciado

Escribir un programa en Python que solicite al usuario un número entero no negativo n y calcule el **n -ésimo término de la sucesión de Fibonacci** utilizando la **fórmula de Binet** (forma cerrada basada en el número de oro). El programa debe mostrar el resultado redondeado al entero más cercano.

Fundamento matemático

La sucesión de Fibonacci se define recursivamente como $F(0) = 0$, $F(1) = 1$ y $F(n) = F(n-1) + F(n-2)$ para $n \geq 2$. Sin embargo, existe una **fórmula cerrada** descubierta por Jacques Philippe Marie Binet en 1843, que permite calcular cualquier término directamente sin recursión ni iteración:

El número de oro (φ)

$$\varphi = (1 + \sqrt{5}) / 2 \approx 1,6180339887...$$

El número de oro aparece en matemáticas, arte y naturaleza. Su conjugado es:

$$\psi = (1 - \sqrt{5}) / 2 \approx -0,6180339887...$$

Fórmula de Binet

$$F(n) = (\varphi^n - \psi^n) / \sqrt{5}$$

Dado que $|\psi| < 1$, el término ψ^n tiende a cero rápidamente, por lo que para n grande se puede aproximar como $F(n) \approx \text{round}(\varphi^n / \sqrt{5})$. En Python, la raíz cuadrada se calcula con el operador `** 0.5` o importando `math.sqrt` (que por ahora no usaremos, ya que no hemos visto importación de módulos).

Ejemplos (casos de análisis y prueba)

n	F(n) exacto	$\varphi^n / \sqrt{5}$	$\text{round}(\varphi^n / \sqrt{5})$	Observación
0	0	0.4472...	0	Caso base
1	1	0.7236...	1	Caso base
5	5	4.9999...	5	Valor pequeño
10	55	55.0000...	55	Valor medio
20	6765	6765.0000...	6765	Valor grande

30	832040	832040.0000...	832040	Límite práctico de precisión float
----	--------	----------------	--------	------------------------------------

Verificación manual para $n = 5$

```

raiz_5 = 5 ** 0.5          # ≈ 2.2360679...
phi    = (1 + raiz_5) / 2  # ≈ 1.6180339...
psi    = (1 - raiz_5) / 2  # ≈ -0.6180339...
phi_n  = phi ** 5          # ≈ 11.0901699...
psi_n  = psi ** 5          # ≈ -0.0901699...
F_5    = (phi_n - psi_n) / raiz_5
        = (11.0901699 - (-0.0901699)) / 2.2360679
        = 11.1803398 / 2.2360679
        ≈ 4.99999...
round(F_5) = 5  ✓

```

Orientaciones para la resolución

Análisis: Una entrada `int` (n), varias constantes intermedias (`raiz_5`, `phi`, `psi`), un cálculo central y un resultado entero. Considerar que para $n = 0$ la fórmula produce 0,447... que se redondea a 0 (correcto).

Diseño: Descomponer la fórmula en pasos intermedios con variables descriptivas. Evitar escribir toda la fórmula en una sola línea: la claridad es más importante que la brevedad.

Codificación: Usar `round()` para obtener el entero más cercano. La función `round()` de Python devuelve un `int` cuando se llama sin segundo argumento.

Reflexión: límites de la precisión float

Para valores de n superiores a aproximadamente 70, la fórmula de Binet con float de doble precisión comienza a producir errores de redondeo que alteran el resultado. Esto es una limitación intrínseca de la representación IEEE 754 estudiada en la clase teórica. En semanas posteriores se verá cómo calcular Fibonacci de forma exacta mediante iteración.

Problema 5: Raíces de una ecuación cuadrática

Enunciado

Escribir un programa en Python que solicite al usuario los coeficientes **a**, **b** y **c** de una ecuación cuadrática de la forma **$ax^2 + bx + c = 0$** y calcule sus dos raíces utilizando la **fórmula general (resolvente)**. El programa debe resolver **todos los casos**: raíces reales distintas ($\Delta > 0$), raíz real doble ($\Delta = 0$) y **raíces complejas conjugadas** ($\Delta < 0$), introduciendo para este último caso el tipo `complex` de Python. El programa debe mostrar ambas raíces con cuatro decimales de precisión.

Fundamento matemático

Toda ecuación de segundo grado con una incógnita puede resolverse mediante la fórmula resolvente, conocida también como fórmula de Bhaskara:

Discriminante

$$\Delta = b^2 - 4ac$$

El discriminante (Δ , delta) determina la naturaleza de las raíces:

$\Delta > 0$: dos raíces reales distintas.

$\Delta = 0$: una raíz real doble (ambas raíces coinciden).

$\Delta < 0$: dos raíces complejas conjugadas (no existen raíces reales).

Fórmula resolvente (caso general)

$$x_1 = (-b + \sqrt{\Delta}) / (2a) \quad x_2 = (-b - \sqrt{\Delta}) / (2a)$$

Cuando $\Delta \geq 0$, la raíz cuadrada del discriminante es un número real y la fórmula produce raíces reales. Pero cuando $\Delta < 0$, la raíz cuadrada de un número negativo no existe en los reales. Aquí entran los **números complejos**.

Números complejos en Python: el tipo complex

Python tiene soporte nativo para números complejos mediante el tipo `complex`. La diferencia con la notación matemática es que Python usa la letra `j` (convención de ingeniería eléctrica) en lugar de `i` (convención matemática) para la unidad imaginaria:

```
# Crear un número complejo directamente
z = 3 + 4j          # parte real = 3, parte imaginaria = 4

# Crear un complejo a partir de sus componentes
z = complex(3, 4)   # equivalente a 3 + 4j

# Acceder a las partes
z.real              # → 3.0
z.imag              # → 4.0

# Conjugado
z.conjugate()       # → (3-4j)
```

La función `complex(parte_real, parte_imag)` permite construir un número complejo a partir de dos valores `float`.

Ejemplos (casos de análisis y prueba)

Caso $\Delta > 0$ — Raíces reales distintas:

a	b	c	Δ	x_1	x_2	Ecuación
1	-5	6	1	3.0000	2.0000	$x^2 - 5x + 6 = 0$
1	0	-4	16	2.0000	-2.0000	$x^2 - 4 = 0$
1	-1	-1	5	1.6180	-0.6180	$x^2 - x - 1 = 0$ (¡número de oro!)

Caso $\Delta = 0$ — Raíz real doble:

a	b	c	Δ	$x_1 = x_2$	Ecuación
1	-6	9	0	3.0000	$x^2 - 6x + 9 = 0$ (= $(x-3)^2$)
4	4	1	0	-0.5000	$4x^2 + 4x + 1 = 0$ (= $(2x+1)^2$)

Caso $\Delta < 0$ — Raíces complejas conjugadas:

a	b	c	Δ	x_1	x_2	Ecuación
1	0	1	-4	$0+1j$	$0-1j$	$x^2 + 1 = 0$
1	2	5	-16	$-1+2j$	$-1-2j$	$x^2 + 2x + 5 = 0$
1	-4	13	-36	$2+3j$	$2-3j$	$x^2 - 4x + 13 = 0$
2	2	1	-4	$-0.5+0.5j$	$-0.5-0.5j$	$2x^2 + 2x + 1 = 0$

Verificación manual: caso real ($x^2 - x - 1 = 0$)

Para $a = 1$, $b = -1$, $c = -1$:

```
discriminante = (-1)**2 - 4*1*(-1) = 1 + 4 = 5      #  $\Delta > 0$ : raíces reales
raiz_disc     = 5 ** 0.5  $\approx$  2.2360679...
x1 = (-(-1) + 2.2360679) / (2*1) = 3.2360679 / 2  $\approx$  1.6180
x2 = (-(-1) - 2.2360679) / (2*1) = -1.2360679 / 2  $\approx$  -0.6180
```

Nótese que $x_1 \approx \phi$ (el número de oro del Problema 4) y $x_2 \approx \psi$ (su conjugado). **Esta conexión entre problemas no es casual:** la ecuación $x^2 - x - 1 = 0$ es precisamente la ecuación característica de la sucesión de Fibonacci.

Verificación manual: caso complejo ($x^2 + 2x + 5 = 0$)

Para $a = 1$, $b = 2$, $c = 5$:

```
discriminante = 2**2 - 4*1*5 = 4 - 20 = -16          # Δ < 0: raíces complejas

# Como Δ < 0, calculamos parte real e imaginaria por separado:
parte_real = -2 / (2*1) = -1.0
abs_disc   = abs(-16)   = 16
parte_imag = 16 ** 0.5 / (2*1) = 4.0 / 2 = 2.0

# Construimos los complejos:
x1 = complex(-1.0, 2.0)  # → (-1+2j)
x2 = complex(-1.0, -2.0) # → (-1-2j)

# Verificación sustituyendo x1 en la ecuación:
# x12 + 2*x1 + 5
# = (-1+2j)**2 + 2*(-1+2j) + 5
# = (1 - 4j + 4j2) + (-2+4j) + 5
# = (1 - 4j - 4) + (-2+4j) + 5      [porque j2 = -1]
# = (-3 - 4j) + (-2 + 4j) + 5
# = 0 + 0j = 0  ✓
```

Verificación manual: el caso emblemático ($x^2 + 1 = 0$)

Para $a = 1$, $b = 0$, $c = 1$. Esta ecuación pregunta: ¿qué número elevado al cuadrado da -1 ?

```
discriminante = 0**2 - 4*1*1 = -4          # Δ < 0
parte_real = -0 / (2*1) = 0.0
parte_imag = abs(-4)**0.5 / (2*1) = 2.0 / 2 = 1.0
x1 = complex(0.0, 1.0)  # → 1j  (= i, la unidad imaginaria)
x2 = complex(0.0, -1.0) # → -1j (= -i)
```

Las raíces son $+i$ y $-i$, que en Python se representan como $1j$ y $-1j$. Este es el ejemplo fundacional de los números complejos: fueron inventados precisamente para poder responder la pregunta $x^2 = -1$.

Orientaciones para la resolución

Análisis: Tres entradas `float` (a , b , c), un cálculo intermedio (discriminante), y según su signo, dos caminos: raíces reales (`float`) o raíces complejas (`complex`). Tres grupos de casos de prueba: $\Delta > 0$ (raíces reales), $\Delta = 0$ (raíz doble), $\Delta < 0$ (raíces complejas). Verificar cruzadamente: el caso $x^2 - x - 1 = 0$ produce el número de oro (Problema 4), y el caso $x^2 + 1 = 0$ produce la unidad imaginaria.

Diseño: Variables: a , b , c (float, entradas); discriminante (float); x_1 y x_2 (float o complex, resultados). Calcular el discriminante como paso intermedio.

Codificación: Usar paréntesis para garantizar la precedencia correcta. La expresión $-b + \text{raiz_disc} / 2 * a$ es **incorrecta** (produce un resultado erróneo). La forma correcta es $(-b + \text{raiz_disc}) / (2 * a)$.

Evaluación: Verificar las raíces sustituyéndolas en la ecuación original: el resultado debe ser (aproximadamente) cero. Para raíces reales: $a * x_1^2 + b * x_1 + c \approx 0$. Para raíces complejas, Python permite la misma operación porque los operadores aritméticos funcionan con complex: $1 * (-1+2j)^2 + 2 * (-1+2j) + 5$ produce $0j$ (que es cero). ✓

Reflexión: precedencia de operadores y la ALU

Los errores de precedencia son uno de los errores semánticos más comunes en programación. A nivel de la ALU, cada operación aritmética se ejecuta una a la vez en un orden determinado. El intérprete Python traduce la expresión respetando las reglas de precedencia del lenguaje, que coinciden (en lo esencial) con las convenciones algebraicas. Los paréntesis permiten al programador explicitar el orden deseado sin depender de la memorización de reglas.

Conexión entre problemas

Los Problemas 4 y 5 están intencionalmente conectados: la ecuación $x^2 - x - 1 = 0$ produce como raíces el número de oro y su conjugado, que son exactamente las constantes usadas en la fórmula de Binet. Identificar estas relaciones entre problemas aparentemente distintos es una habilidad central del pensamiento algorítmico.

Conexión con Tipos de Datos: el tercer tipo numérico

En la clase teórica se presentaron los tres tipos numéricos nativos de Python: int, float y complex. Los Problemas 1–4 trabajan con int y float. Este problema introduce complex de forma natural y motivada: los números complejos no son una curiosidad abstracta, sino la única forma de resolver completamente una ecuación cuadrática. A nivel de la ALU, las operaciones con complejos se implementan como pares de operaciones con float (parte real e imaginaria por separado), lo que duplica el trabajo computacional.

Notas finales

Estos cinco problemas cubren los contenidos prácticos de la **Semana 1** del curso: entrada/salida, asignación, operaciones aritméticas con tipos `int`, `float` y `complex`, construcción de cadenas (`str`), y traza manual de algoritmos secuenciales. Los problemas están ordenados en complejidad creciente y presentan conexiones cruzadas deliberadas (Problemas 4 y 5).

Criterios de evaluación formativa

Para cada problema, se valorará la aplicación completa de la metodología de Pólya: análisis explícito con casos de prueba (no solo los proporcionados), diseño mediante pseudocódigo o comentarios estructurados, codificación limpia que respete el modelo de programa del curso, y evaluación con verificación de resultados. Se espera documentación interna (*literate programming*) que explique el proceso y las decisiones tomadas.

Para profundizar

Una vez resueltos los cinco problemas, considerar estas extensiones como desafío adicional: (1) ¿qué modificaciones necesitaría el Problema 1 para convertir de Fahrenheit a Celsius? (inversión de fórmula); (2) ¿cómo se podría extender el Problema 3 para incorporar también los segundos, y cuál sería el término adicional en la fracción del día? (pista: un día tiene 86400 segundos); (3) comparar el resultado de la fórmula de Binet con la secuencia calculada manualmente para $n = 0, 1, 2, \dots, 10$. Estas extensiones anticipan contenidos de las próximas semanas.